CAIRO SECURITY CLAN

WORLDCOIN ID BRIDGE

SECURITY ASSESMENT REPORT

MAY 2025

Prepared for

WORLDCOIN

Contents

1 About Cairo Security Clan

Cairo Security Clan is a leading force in the realm of blockchain security, dedicated to fortifying the foundations of the digital age. As pioneers in the field, we specialize in conducting meticulous smart contract security audits, ensuring the integrity and reliability of decentralized applications built on blockchain technology.

At Cairo Security Clan, we boast a multidisciplinary team of seasoned professionals proficient in blockchain security, cryptography, and software engineering. With a firm commitment to excellence, our experts delve into every aspect of the Web3 ecosystem, from foundational layer protocols to application-layer development. Our comprehensive suite of services encompasses smart contract audits, formal verification, and real-time monitoring, offering unparalleled protection against potential vulnerabilities.

Our team comprises industry veterans and scholars with extensive academic backgrounds and practical experience. Armed with advanced methodologies and cutting-edge tools, we scrutinize and analyze complex smart contracts with precision and rigor. Our track record speaks volumes, with a plethora of published research papers and citations, demonstrating our unwavering dedication to advancing the field of blockchain security.

At Cairo Security Clan, we prioritize collaboration and transparency, fostering meaningful partnerships with our clients. We believe in a customer-oriented approach, engaging stakeholders at every stage of the auditing process. By maintaining open lines of communication and soliciting client feedback, we ensure that our solutions are tailored to meet the unique needs and objectives of each project.

Beyond our core services, Cairo Security Clan is committed to driving innovation and shaping the future of blockchain technology. As active contributors to the ecosystem, we participate in the development of emerging technologies such as Starknet, leveraging our expertise to build robust infrastructure and tools. Through strategic guidance and support, we empower our partners to navigate the complexities of the blockchain landscape with confidence and clarity.

In summary, Cairo Security Clan stands at the forefront of blockchain security, blending technical prowess with a client-centric ethos to deliver unparalleled protection and peace of mind in an ever-evolving digital landscape. Join us in safeguarding the future of decentralized finance and digital assets with confidence and conviction.

2 Disclaimer

Disclaimer Limitations of this Audit:

This report is based solely on the materials and documentation provided by you to Cairo Security Clan for the specific purpose of conducting the security review outlined in the Summary of Audit and Scoped Files. The findings presented here may not be exhaustive and may not identify all potential vulnerabilities. Cairo Security Clan provides this review and report on an "as-is" and "as-available" basis. You acknowledge that your use of this report, including any associated services, products, protocols, platforms, content, and materials, occurs entirely at your own risk.

Inherent Risks of Blockchain Technology:

Blockchain technology remains in its developmental stage and is inherently susceptible to unknown risks and vulnerabilities. This review is specifically focused on the smart contract code and does not extend to the compiler layer, programming language elements beyond the reviewed code, or other potential security risks outside the code itself.

Report Purpose and Reliance:

This report should not be construed as an endorsement of any specific project or team, nor does it guarantee the absolute security of the audited smart contracts. No third party should rely on this report for any purpose, including making investment or purchasing decisions.

Liability Disclaimer:

To the fullest extent permitted by law, Cairo Security Clan disclaims all liability associated with this report, its contents, and any related services and products arising from your use. This includes, but is not limited to, implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

Third-Party Products and Services:

Cairo Security Clan does not warrant, endorse, guarantee, or assume responsibility for any products or services advertised by third parties within this report, nor for any open-source or third-party software, code, libraries, materials, or information linked to, referenced by, or accessible through this report, its content, and related services and products. This includes any hyperlinked websites, websites or applications appearing on advertisements, and Cairo Security Clan will not be responsible for monitoring any transactions between you and third-party providers. It is recommended that you exercise due diligence and caution when considering any third-party products or services, just as you would with any purchase or service through any medium.

Disclaimer of Advice:

FOR THE AVOIDANCE OF DOUBT, THIS REPORT, ITS CONTENT, ACCESS, AND/OR USE, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHOULD NOT BE CONSIDERED OR RELIED UPON AS FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER PROFESSIONAL ADVICE.

3 Executive Summary

This document presents the security review performed by Cairo Security Clan on the Worldcoin ID.

Worldcoin ID is a privacy-preserving digital identity protocol that enables Sybil-resistant verification using zero-knowledge proofs. Part of the Worldcoin ecosystem, it allows users to prove their uniqueness as humans without revealing personal data. It's increasingly used as a trust layer in decentralized applications. Learn more from docs.

The audit was performed using

- manual analysis of the codebase,
- automated analysis tools,
- simulation of the smart contract,
- analysis of edge test cases

6 points of attention, where 0 is classified as Critical, 0 is classified as High, 0 is classified as Medium, 1 is classified as Low, 2 are classified as Informational, 2 are classified as Best Practices and 1 is classified as Undetermined. The issues are summarized in Fig. 1.

This document is organized as follows. Section 1 About Cairo Security Clan. Section 2 Disclaimer. Section 3 Executive Summary. Section 4 Summary of Audit. Section 5 Risk Classification. Section 6 Issues by Severity Levels. Section 7 Test Evaluation.

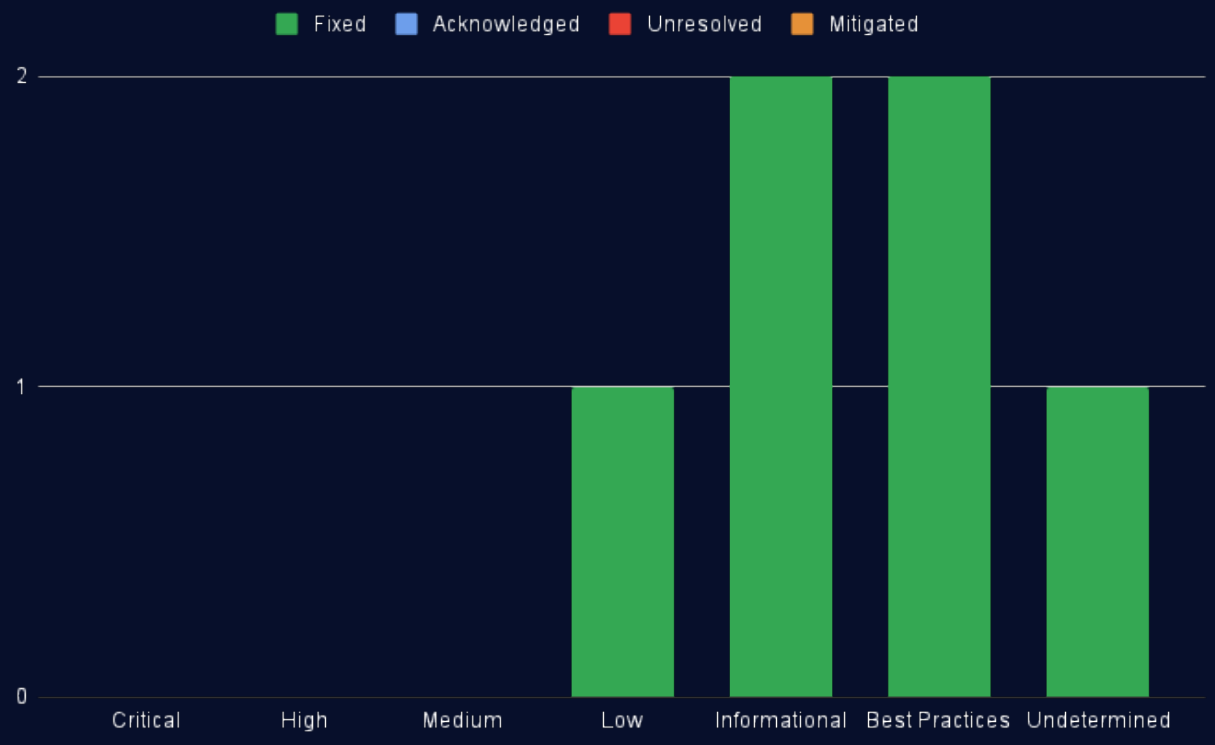

Fig 1: Distribution of issues: Critical (0), **High** (0), **Medium** (0), **Low** (1), **Informational** (2), **Best Practices** (2), **Undetermined** (1). **Distribution of status: Fixed** (6), **Acknowledged** (0), **Mitigated** (0), **Unresolved** (0).

4 Overview

The Worldcoin ID Bridge is a cross-chain identity verification protocol, It facilitates secure, privacy-preserving identity verification between World Chain and StarkNet. By bridging Merkle tree roots and leveraging zero-knowledge proofs, it enables users to prove membership in the Worldcoin identity set on StarkNet without exposing information. The protocol maintains a root history with expiry to ensure both security and up-to-date state synchronization with main chain.

Our review primarily focused on the security aspects of these key components/contracts :

- **StarkWorldID**: Serves as the bridge interface layer on StarkNet side, implementing L1 handlers for receiving root updates and managing bridge configuration through authorized WorldChain messages.
- **WorldID**: It serves as the foundation of the Starknet side bridge, implementing Groth16-based zero-knowledge proof verification and managing a time-bounded Merkle root history system. It ensures secure, privacy-preserving identity verification by validating proofs against synchronized roots from the WorldChain.
- **StarkStateBridge**: It serves as the World Chain bridge component, facilitating root propagation to StarkNet and managing administrative operations. It enables secure cross-chain communication by sending roots and configuration changes to Starknet side bridge , while enforcing owner-only access controls and configurable bridge parameters.
- **CrossDomainOwnable**: Manages ownership permissions where only messages originating from the authorized WorldChain address can execute privileged functions on the StarkNet WorldID contract, with the ability to securely transfer this ownership to another WorldChain address.

5 Summary of Audit

Audit Type	Security Review
Cairo Version	2.8.5
Solidity Version	0.8.30
Final Report	18/06/2025
Repository	NethermindEth/starknet-worldcoin-bridge
Initial Commit Hash	20ea54f9ba4757a6ba7916187740127f4e389680
Final Commit Hash	8b220a543c522c136ba9fd24e1aadcefc516a789
Documentation	Website documentation
Test Suite Assessment	High

5.1 Scoped Files

	Contracts
1	state_bridge_l1/src/StarkStateBridge.sol
2	state_bridge_l1/src/interfaces/IRootHistory.sol
3	state_bridge_l1/src/interfaces/IStarkWorldID.sol
4	state_bridge_l1/src/interfaces/IWorldIDIdentityManager.sol
5	state_bridge_l2/world_id_state_bridge/src/lib.cairo
6	state_bridge_l2/world_id_state_bridge/src/stark_world_id.cairo
7	state_bridge_l2/world_id_state_bridge/src/stark_world_id/cross_domain_ownable.cairo
8	state_bridge_l2/world_id_state_bridge/src/stark_world_id/interface_stark_world_id.cairo
9	state_bridge_l2/world_id_state_bridge/src/stark_world_id/world_id_bridge.cairo
10	state_bridge_l2/world_id_state_bridge/src/stark_world_id/cross_domain_ownable/cross_domain_ownable.cairo
11	state_bridge_l2/world_id_state_bridge/src/stark_world_id/cross_domain_ownable/interface_cross_domain_ownable.cairo
12	state_bridge_l2/world_id_state_bridge/src/stark_world_id/world_id_bridge/groth16_verifier_constants.cairo
13	state_bridge_l2/world_id_state_bridge/src/stark_world_id/world_id_bridge/interface_world_id.cairo
14	state_bridge_l2/world_id_state_bridge/src/stark_world_id/world_id_bridge/semaphore_tree_depth_validator.cairo
15	state_bridge_l2/world_id_state_bridge/src/stark_world_id/world_id_bridge/world_id_bridge.cairo

5.2 Issues

	Findings	Severity	Update
1	Missing value validation in payable functions	Low	Fixed
2	Contradictory documentation for ZK-Proof verification failure behavior	Informational	Fixed
3	Incorrect Natspec comment	Informational	Fixed
4	Confusing naming of `owner`	Best Practices	Fixed
5	Typo	Best Practices	Fixed
6	Significant deviations from Garaga reference in ZK-Proof verification	Undetermined	Fixed

6 Risk Classification

The risk rating methodology used by Cairo Security Clan follows the principles established by the CVSS risk rating methodology. The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely an attacker will uncover and exploit the finding. This factor will be one of the following values:

a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;

b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;

c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to Motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

a) **High**: The issue can cause significant damage such as loss of funds or the protocol entering an unrecoverable state;

b) **Medium**: The issue can cause moderate damage such as impacts that only affect a small group of users or only a particular part of the protocol;

c) **Low**: The issue can cause little to no damage such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Likelihood		
		High	**Medium**	**Low**
Impact	**High**	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Info/Best Practices

To address issues that do not fit a High/Medium/Low severity, Cairo Security Clan also uses three more finding severities: **Informational**, **Best Practices** and **Gas**

a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to formally pass to the client;

b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;

c) **Gas** findings are used when some piece of code uses more gas than it should be or have some functions that can be removed to save gas.

7 Issues by Severity Levels

7.1 Low

7.1.1 Missing value validation in payable functions

File(s): state_bridge_l1/src/StarkStateBridge.sol

Description: Several functions in `StarkStateBridge` are marked as `external payable` but do not validate the `msg.value` against the actual amount used for cross-chain messaging. These functions include:

```
function propagateRoot() external payable {
    // Uses _gasLimitPropagateRoot for the value parameter
}

function transferOwnershipStark(address _owner) external payable onlyOwner {
    // Uses _gasLimitTransferOwnership for the value parameter
}

function setRootHistoryExpiry(uint256 _rootHistoryExpiry) external payable onlyOwner {
    // Uses _gasLimitSetRootHistoryExpiry for the value parameter
}
```

Each function accepts ETH but uses a predefined gas limit parameter instead of the provided `msg.value`. While excess funds aren't permanently locked (they remain in the contract's balance for future calls), this design creates:

- Inconsistent user experience where users might send incorrect amounts
- Potential for users to unintentionally subsidize other users' transactions
- Risk of transaction failure if insufficient ETH is sent

Recommendation(s): Consider adding validation to ensure the correct amount of ETH is sent. For example:

```
function propagateRoot() external payable {
    require(msg.value >= _gasLimitPropagateRoot, "Insufficient value");
    // Function logic
}
```

Status: Fixed

Update from the client: Fixed in commit 30f7ebe.

7.2 Informational

7.2.1 Contradictory documentation for ZK-Proof verification failure behavior

File(s): state_bridge_l2/world_id_state_bridge/src/stark_world_id/world_id_bridge/world_id_bridge.cairo

Description: There is a contradiction between the documentation and the implementation of the `verify_groth16_proof_bn254()` function regarding its behavior when a proof verification fails:

```
/// Either panics or succeeds on verification
```

This docstring claims the function will either panic (on failure) or succeed (on verification). However, the implementation comments and actual code suggest different behavior:

```
// This function returns an Option for the public inputs if the proof is valid.
// If the proof is invalid, the execution will either fail or return None.

// [...]

if check == true {
    return Option::Some(groth16_proof.public_inputs);
} else {
    return Option::None;
}
```

The implementation returns `Option::None` for invalid proofs rather than panicking. This discrepancy could cause integration issues if developers rely on the docstring's claim that the function will panic on verification failure.

Recommendation(s): Consider modifying the implementation to match the documented behavior by replacing the return of `Option::None` with a panic.

Status: Fixed

Update from the client: Fixed in commit 5525c29c.

7.2.2 Incorrect Natspec comment

File(s): state_bridge_l1/src/StarkStateBridge.sol

Description: The Natspec comment for the function `setGasLimitTransferOwnership()` is incorrectly describing the function `setGasLimitSetRootHistoryExpiry()` instead. The comment references *"SetRootHistoryExpiry"* when it should be referring to *"TransferOwnership"*. This mismatch between documentation and implementation can lead to confusion during code review and maintenance.

```
/// @notice Sets the gas limit for the SetRootHistoryExpiry method
/// @param _starkGasLimit The new gas limit for the SetRootHistoryExpiry method
function setGasLimitTransferOwnership(uint256 _starkGasLimit) external onlyOwner {
    if (_starkGasLimit <= 0) {
        revert GasLimitZero();
    }

    _gasLimitTransferOwnership = _starkGasLimit;

    emit SetGasLimitTransferOwnership(_starkGasLimit);
}
```

Recommendation(s): Consider updating the Natspec comment to correctly describe the `setGasLimitTransferOwnership()` function.

Status: Fixed

Update from the client: Fixed in commit e17a4bf9.

7.3 Best Practices

7.3.1 Confusing naming of `owner`

File(s): state_bridge_l2/world_id_state_bridge/src/stark_world_id/cross_domain_ownable/cross_domain_ownable.cairo

Description: Both the L2 and L1 contracts misuse the term `owner` to refer to the address of the `StarkStateBridge` contract, not an actual ownership role. This is misleading and can cause misunderstandings around access control and contract behavior.

In the Cairo L2 contract:

```
fn _initialize(ref self: ComponentState<TContractState>, owner: EthAddress) {
    self.emit(NewL1Owner { previous_l1_owner: self.l1_owner.read(), new_l1_owner: owner });
    self.l1_owner.write(owner); // @audit It is not owner but the address of StarkStateBridge. Confusing naming
}
```

Here, the `owner` parameter is used to store the `StarkStateBridge` address in `l1_owner`, and the emitted event `NewL1Owner` further reinforces the confusion by implying a change in ownership.

Similarly, in the Solidity L1 contract:

```
// @audit This function is not ownership transfer, it is changing the StarkStateBridge address on L2
function transferOwnershipStark(address _owner) external payable onlyOwner {
    ...
    // @audit Event should emit current StarkStateBridge address and new StarkStateBridge address instead (
     address(this), _owner)
    emit OwnershipTransferredStark(owner(), _owner);
}
```

The `transferOwnershipStark()` function does not transfer L1 ownership but instead initiates a change to the `StarkStateBridge` address on L2. Naming both the function and emitted event around "ownership" misrepresents the action being performed.

Recommendation(s): Consider renaming misleading identifiers such as `owner`, `transferOwnershipStark`, and event names like `OwnershipTransferredStark` to clearly reflect their actual role.

Status: Fixed

Update from the client: Fixed in commit 8b220a5.

7.3.2 Typo

File(s): state_bridge_l1/src/StarkStateBridge.sol

Description: There is a duplicate statement in the `setStarkWorldIDAddress()` function. After emitting the `SetStarkWorldIDAddress` event with the parameter `_starkWorldIDAddress`, there's an additional redundant statement `(_starkWorldIDAddress);` that serves no purpose and should be removed. This type of typo indicates the code may not have been thoroughly reviewed before deployment.

```
function setStarkWorldIDAddress(uint256 _starkWorldIDAddress) external onlyOwner {
    // ...
    emit SetStarkWorldIDAddress(_starkWorldIDAddress);(_starkWorldIDAddress); // @audit typo
}
```

Recommendation(s): Consider removing the redundant statement `(_starkWorldIDAddress);` after the event emission.

Status: Fixed

Update from the client: Fixed in commit 30f7ebe.

7.4 Undetermined

7.4.1 Significant deviations from Garaga reference in ZK-Proof verification

File(s): state_bridge_l2/world_id_state_bridge/src/stark_world_id/world_id_bridge/world_id_bridge.cairo

Description: The `verify_groth16_proof_bn254()` function deviates significantly from the Garaga reference implementation in several critical ways:

- Different `ECIP_OPS_CLASS_HASH` constant value
- Different function selector: `msm_g1_u288` vs. `msm_g1` in the reference
- Different MSM calldata preparation approach
- Added application-specific logic (`require_valid_root` check)
- No clear documentation justifying these modifications

These differences could potentially affect the security and correctness of the cryptographic verification process, but a comprehensive cryptographic assessment is required to determine the actual impact.

Recommendation(s): Either align the implementation with the Garaga reference or thoroughly document and justify each deviation. Implement comprehensive testing with known valid and invalid proofs to ensure correctness.

Status: Fixed

Update from the client: Fixed in commit 30f7ebe.

8 Test Evaluation

8.1 L1 Compilation Output

```
forge build
Compiling...
Compiling 39 files with Solc 0.8.30
Solc 0.8.30 finished in 535.64ms
Compiler run successful!
```

8.2 L2 Compilation Output

```
scarb build
    Finished `release` profile [optimized] target(s) in 2m 13s
   Compiling world_id_state_bridge v0.1.0 (world_id_state_bridge/Scarb.toml)

    Finished `dev` profile target(s) in 3 minutes
```

8.3 L1 Tests Output

```
forge test
Compiling...
Compiling 1 files with Solc 0.8.30
Solc 0.8.30 finished in 937.54ms
Compiler run successful!

Ran 10 tests for test/TestUnitStarkStateBridge.t,.sol:UnitStarkStateBridge
[PASS] testFail_SetStarkWorldIDAddress() (gas: 7673)
[PASS] testFail_SetStarknetCoreContract() (gas: 7777)
[PASS] testFail_SetWorldIDIdentityManager() (gas: 7755)
[PASS] testFail_TransferOwnershipStark() (gas: 7863)
[PASS] test_SetFeeLimitPropagateRoot() (gas: 17622)
[PASS] test_SetFeeLimitSetRootHistoryExpiry() (gas: 17721)
[PASS] test_SetFeeLimitTransferOwnership() (gas: 17623)
[PASS] test_SetStarkWorldIDAddress() (gas: 17632)
[PASS] test_SetStarknetCoreContract() (gas: 17835)
[PASS] test_SetWorldIDIdentityManager() (gas: 17792)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 12.25ms (2.81ms CPU time)

Ran 1 test suite in 92.79ms (12.25ms CPU time): 10 tests passed, 0 failed, 0 skipped (10 total tests)
```

8.4 L2 Tests Output

```
snforge test
warn: found duplicate packages named `trace_info v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=
    v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)`

Found locations:
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866

Because of this, referencing package `trace_info v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=
    v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)` will fail.

warn: found duplicate packages named `addition v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0
    .44.0#437d128b4f54847666ea8021530d237f6bf47866)`

Found locations:
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866

Because of this, referencing package `addition v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0
    .44.0#437d128b4f54847666ea8021530d237f6bf47866)` will fail.

```

```
warn: found duplicate packages named `fibonacci v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)`

Found locations:
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866

Because of this, referencing package `fibonacci v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)` will fail.

warn: found duplicate packages named `hello_workspaces v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)`

Found locations:
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866

Because of this, referencing package `hello_workspaces v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)` will fail.

warn: found duplicate packages named `trace_info v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)`

Found locations:
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866

Because of this, referencing package `trace_info v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)` will fail.

warn: found duplicate packages named `addition v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)`

Found locations:
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866

Because of this, referencing package `addition v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)` will fail.

warn: found duplicate packages named `fibonacci v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)`

Found locations:
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866

Because of this, referencing package `fibonacci v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)` will fail.

warn: found duplicate packages named `hello_workspaces v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)`

Found locations:
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866
- git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866

Because of this, referencing package `hello_workspaces v0.1.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)` will fail.

   Compiling snforge_scarb_plugin v0.44.0 (git+https://github.com/foundry-rs/starknet-foundry?tag=v0.44.0#437d128b4f54847666ea8021530d237f6bf47866)
    Finished `release` profile [optimized] target(s) in 0.04s
   Compiling test(world_id_state_bridge_unittest) world_id_state_bridge v0.1.0 (/starknet-worldcoin-bridge/state_bridge_l2/world_id_state_bridge/Scarb.toml)
    Finished `dev` profile target(s) in 13 seconds


Collected 10 test(s) from world_id_state_bridge package
Running 10 test(s) from src/
```

```
[PASS] world_id_state_bridge::tests::stark_world_id::world_id_bridge::tests::test_valid_overwrite_latest_root (l1_gas: ~0, l1_data_gas: ~448, l2_gas: ~292160)
[PASS] world_id_state_bridge::tests::stark_world_id::world_id_bridge::tests::test_set_root_expiry (l1_gas: ~0, l1_data_gas: ~256, l2_gas: ~110720)
[PASS] world_id_state_bridge::tests::stark_world_id::world_id_bridge::tests::test_constructor_with_invalid_tree_depth (l1_gas: ~0, l1_data_gas: ~160, l2_gas: ~40000)
[PASS] world_id_state_bridge::tests::stark_world_id::tests::test_invalid_only_owner_receive_root (l1_gas: ~0, l1_data_gas: ~352, l2_gas: ~120960)
[PASS] world_id_state_bridge::tests::stark_world_id::world_id_bridge::tests::test_latest_root (l1_gas: ~0, l1_data_gas: ~448, l2_gas: ~372160)
[PASS] world_id_state_bridge::tests::stark_world_id::world_id_bridge::tests::test_get_tree_depth (l1_gas: ~0, l1_data_gas: ~256, l2_gas: ~80000)
[PASS] world_id_state_bridge::stark_world_id::world_id_bridge::semaphore_tree_depth_validator::tests::test_valid_depth (l1_gas: ~0, l1_data_gas: ~0, l2_gas: ~40000)
[PASS] world_id_state_bridge::tests::stark_world_id::tests::test_only_owner_receive_root (l1_gas: ~0, l1_data_gas: ~544, l2_gas: ~247040)
[PASS] world_id_state_bridge::tests::stark_world_id::world_id_bridge::tests::test_valid_require_valid_root (l1_gas: ~0, l1_data_gas: ~448, l2_gas: ~402880)
[PASS] world_id_state_bridge::tests::stark_world_id::tests::test_invalid_constructor_root (l1_gas: ~0, l1_data_gas: ~288, l2_gas: ~200000)
Tests: 10 passed, 0 failed, 0 skipped, 0 ignored, 0 filtered out
```